

# My SimpleImputer Study Q&A

Questions I asked — and the answers that clicked for me

---

## Q1 — What are the parameters of SimpleImputer?

I didn't know what options I could pass in when creating a SimpleImputer. Here is the full signature with all parameters:

```
SimpleImputer(  
    missing_values = np.nan,  
    strategy = 'mean',  
    fill_value = None,  
    copy = True,  
    add_indicator = False,  
    keep_empty_features = False  
)
```

| Parameter           | Default | What it does                                    |
|---------------------|---------|-------------------------------------------------|
| missing_values      | np.nan  | What value to treat as 'missing'                |
| strategy            | 'mean'  | mean / median / most_frequent / constant        |
| fill_value          | None    | The constant to use when strategy='constant'    |
| copy                | True    | If False, modifies data in-place (saves memory) |
| add_indicator       | False   | Adds a 0/1 column marking where NaN was         |
| keep_empty_features | False   | Keep columns that are 100% missing              |

## Q2 — How exactly does fit() learn? How exactly does transform() work?

I wanted to know what happens internally — not just 'it learns the mean' but what that actually means step by step.

### How fit() learns:

It runs simple math on each column, ignoring NaN values, and stores the result in `imputer.statistics_`:

```
strategy='mean' → statistics_[i] = np.nanmean(column[i])  
strategy='median' → statistics_[i] = np.nanmedian(column[i])  
strategy='most_frequent' → statistics_[i] = scipy.stats.mode(column[i])  
strategy='constant' → statistics_[i] = fill_value
```

After `fit()`, you can inspect what was learned:

```
print(imputer.statistics_) # e.g. [30.0, 52000.0]  
print(imputer.n_features_in_) # how many columns it saw
```

### How transform() works:

It loops through every cell. If the cell equals missing\_values, it swaps it with the value stored in statistics\_. Everything else is left alone:

```
For each column i:
  For each row j:
    if X[j][i] == missing_values:
      X[j][i] = statistics_[i] # replace
    else:
      leave it alone # untouched
```

### Parameters for fit() and transform():

| Method       | Param | Description                                           |
|--------------|-------|-------------------------------------------------------|
| fit(X, y)    | X     | Training data to learn from (array or DataFrame)      |
| fit(X, y)    | y     | Ignored — only exists for sklearn API consistency     |
| transform(X) | X     | Data to fill in — must have same columns as fit() saw |

### Q3 — I still don't understand how fit() and transform() work!

The technical explanation wasn't clicking. I needed a real-life analogy.

#### The Substitute Teacher Analogy:

|                    |                                                 |                                                                                   |
|--------------------|-------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>fit()</b>       | Taking attendance<br>Learning the class average | You look at all grades and write down: 'average = 75'. Nothing changed yet. You j |
| <b>transform()</b> | Filling in absent students' grades              | As a student was absent (missing value). You open your notes, see average=75, and |

#### Step-by-step in code:

```
X = [[25], [30], [35], [np.nan]]

# fit() — looks at [25, 30, 35], ignores NaN, calculates mean=30.0
imputer.fit(X)
# imputer.statistics_ = [30.0] ← stored, nothing changed

# transform() — finds NaN, replaces with 30.0
X_clean = imputer.transform(X)
# Result: [[25], [30], [35], [30.0]]
```

Key rule: fit() only on training data — because statistics\_ must reflect training data's mean, not the test set's. Fitting on test = data leakage!

### Q4 — Does transform() only work with Null/NaN values?

I assumed transform() specifically looked for NaN. The real answer: it looks for whatever missing\_values is set to.

```
# Default — looks for NaN
SimpleImputer(missing_values=np.nan)
```

```
# You can change it to anything
SimpleImputer(missing_values=-1) # -1 means unknown
SimpleImputer(missing_values=0) # 0 means unknown
SimpleImputer(missing_values='unknown') # string missing marker
```

### Example with -1 as the missing marker:

```
X_train = [[25], [30], [35], [-1]]

imputer = SimpleImputer(missing_values=-1, strategy='mean')
imputer.fit(X_train)
# Learns mean of [25,30,35] = 30.0 (ignores -1)

imputer.transform([[ -1], [28]])
# [[-1] becomes 30.0]
# [[28] stays 28 ]
# Result: [[30.0], [28.0]]
```

transform() rule: if cell == missing\_values → replace. Otherwise → leave alone. It is just a targeted find-and-replace. Nothing more.

## O5 — My final understanding (the moment it clicked!)

After all the questions, I summarized it myself and it was confirmed correct:

```
imputer = SimpleImputer(strategy='mean')
# Step 1: Define the strategy – what to calculate. Nothing happens yet.

imputer.fit(X_train)
# Step 2: Calculate mean of each column, store in statistics_. Data unchanged.

X_clean = imputer.transform(X_test)
# Step 3: Find cells == missing_values, replace with statistics_. Done.
```

|                        |                                            |
|------------------------|--------------------------------------------|
| <b>SimpleImputer()</b> | Define WHAT to calculate (strategy)        |
| <b>fit()</b>           | CALCULATE it and store in statistics_      |
| <b>transform()</b>     | FIND missing_values, SWAP with statistics_ |

The one rule to never forget: fit() ONLY on training data. transform() on both train and test using the same statistics\_.